From Static to Quasi-Dynamic: Reconsidering Scheduling and Memory in SDF Compilers

Pedro Ciambra*†, Anaëlle Cloarec†, Hervé Yviquel*, Mickaël Dardaillon†, Maxime Pelcat†
*Instituto de Computação, Universidade Estadual de Campinas (UNICAMP)
Campinas, São Paulo, Brazil
pedro.ciambra@ic.unicamp.br, herve.yviquel@ic.unicamp.br
†Univ Rennes, INSA Rennes, CNRS, IETR – UMR 6164
F-35000 Rennes, France
anaelle.cloarec@insa-rennes.fr, mickael.dardaillon@insa-rennes.fr, maxime.pelcat@insa-rennes.fr

***Abstract*—Synchronous Dataflow (SDF) is a popular Model of Computation (MoC) for signal processing applications, thanks to its static analizability and provably deadlock-free execution. SDF compilers have traditionally prioritized static schedulers for their low runtime latency, particularly for signal processing and embedded systems. However, the conventional static technique of converting of multi-rate graphs to single-rate equivalents introduces scalability challenges through exponential graph growth and lost coarse-grained symmetry information. We revisit the idea of runtime scheduling for static dataflow and explore a tradeoff between latency, binary size and compilation time.**

Our approach introduces a specialized "quasi-dynamic" runtime scheduler and memory manager that leverages static topology information to precompute buffer sizes and allows out-of-order execution of actor firings, while thread assignment and memory allocation are handled at runtime through the OpenMP runtime and standard allocation primitives (*malloc*). By preserving multi-rate actor relationships, we avoid the node explosion inherent to single-rate conversion and improve the feasibility of SDF for a previously unreachable class of workloads. Comparative analysis of the current prototype against the Preesm static scheduler on the *degridder* radio astronomy application show, on average, 12.2x compilation time speedup and 4.4x smaller binary sizes, while only suffering 1.4x slowdown.

***Index Terms*—dataflow, SDF, runtime, scheduling, static, quasi-static, quasi-dynamic.**

I. Introduction

Synchronous Dataflow (SDF) models have long been favored in signal processing and embedded systems due to their analyzability and compile-time optimizations. The ability to statically schedule actor firings and preallocate memory buffers makes SDF an attractive choice for applications where latency and memory footprint are critical. As a result, the dataflow compilation landscape has historically prioritized static analysis and single-rate transformations, often at the expense of scalability and generality [1].

However, this preference for static techniques reveals significant limitations when applied to large or irregular applications. The transformation of multi-rate SDF graphs into single-rate equivalents, while enabling precise scheduling, can lead to combinatorial growth in graph size, bloated binaries, and excessive compilation times [2]. This challenge becomes particularly acute in domains such as computer vision or scientific computing, where regular structures and coarse-grained parallelism are common, but cannot be efficiently exploited using traditional static strategies.

Meanwhile, the broader systems community — especially in the context of managed runtimes and compilers — has embraced runtime scheduling and memory allocation as effective tools to balance performance and generality [3] [4]. In contrast, the SDF compiler community has yet to fully explore these techniques, particularly when it comes to dynamic memory management optimized for the specific semantics of dataflow.

In this work, we revisit dynamic scheduling and allocation for SDF from a fresh perspective. Rather than discarding the advantages of static information, we propose a *quasi-dynamic* approach: one that uses compile-time graph structure to inform a runtime memory management layer capable of out-of-order execution, fine-grained allocation, and reduced memory footprint. Central to our proposal is the *Virtual FIFO*, a memory management system that retains FIFO semantics while allowing memory reuse across edges and decoupling execution dependencies from memory layout.

Preliminary experiments on the *degridder* radio astronomy application suggest that this hybrid strategy provides more scalable and memory-efficient compilation pipelines without incurring the overheads traditionally associated with fully dynamic runtimes. Our experiments demonstrate clear benefits in terms of binary size and compilation time, being able to tackle previously intractable graphs and suggesting that it is time to rethink the strict separation between static and dynamic techniques in dataflow compilation.

This article is structured as follows. In Section II, we provide background on dataflow models of computation and their different scheduling strategies, examples of implementation of which are provided in Section III. Section IV covers the limitations of these strategies when applied to static dataflow, and our solution is detailed in Section V. Sections VI and VII detail our chosen experiment target application and the results thereof obtained. Section VIII sums up the final conclusions of this work and outline the next steps for this research.

II. Background

In this section, we introduce the research field of dataflow, with a focus on the differences between dynamic and static scheduling.

Dataflow is a family of MoCs that provides an abstract representation of parallel programs, sharing similarities with paradigms such as *stream processing* and *reactive programming* [5]. Its independence from specific parallelization mechanisms allows for broad applicability, in areas such as hardware design and machine learning. The concept of representing data transformations as flow graphs has a long history, emerging in hardware research during the 1970s and 1980s and appearing as a programming paradigm as early as 1960 [6]. Additionally, due to its compatibility with graphical representation, dataflow has been a subject of interest in visual programming research [7].

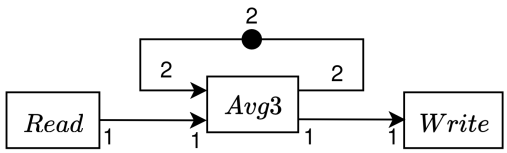

Fig. 1. An example dataflow graph (specifically, an SDF graph implementation of a low-pass filter). The numbers denote the amount of tokens consumed/produced by each edge at each firing of an actor, and the black circle represents a delay of 2 initial tokens (provided by the programmer and present before the first actor firing).

Dataflow models represent algorithms as directed graphs, as illustrated in Fig. 1. In these graphs, nodes (referred to as "actors") correspond to processing steps, while edges represent data queues (first-in-first-out data structures, or FIFOs) that store data between processing steps. Each actor encapsulates an operation that can be executed (or "fired") to perform work. Before firing, an actor consumes values (referred to as "tokens") from its input edges, and inserts tokens into its output edges after the computation is finished. These firing actions are treated as indivisible units of work and are typically modeled as black-box operations. By scheduling the firing of actors in an appropriate order, data undergoes transformations as it moves from edge to edge, ultimately reaching the sink nodes as the final output of the algorithm. This representation captures the data dependencies between different parts of the algorithm, simplifying the identification of parallelizable regions that can then be assigned to the available processing elements (PEs). This task can be automated, either dynamically by a runtime system or statically during compilation.

Dataflow is an architecture-agnostic representation and does not encode memory information; as long as data dependencies are respected, there is no expectation that tokens be stored contiguously in memory. Because this allows pipelining and streaming, dataflow can be used in memory-constrained platforms such as Digital Signal Processors (DSPs) and Field Programmable Gate Arrays (FPGAs).

Dataflow systems can be implemented at varying levels of restrictiveness, trading off design flexibility with statically-known information for compile-time analysis and optimization. Many dataflow MoCs have been defined over the years [8]; in order to focus on the problem at hand, only a subset is covered in this article.

A. Dynamic Models of Computation

The most flexible MoCs are capable of dynamically adapting the behavior of their actors to the data contained within the tokens, which is necessary in contexts where the amount of computation is data-dependent. While this increases programmer and algorithm flexibility, it also limits our ability to generate optimized schedules ahead of time; in the context of shared memory architectures, executing such models requires robust and overhead-heavy runtime mechanisms for effective scheduling and memory management [9]. One example of dynamic MoC, Dataflow Process Networks (DPNs), has the property of allowing non-deterministic algorithms; this is useful for situations that deal with unpredictability, such as resource management problems like load balancing. Kahn Process Networks (KPNs) [10] are a more constrained MoC where the relative order of actor execution is always the same for a given input, providing a level of predictability that enables a moderate level of static analysis and optimizations, as well as being more suitable for hardware synthesis. These MoCs have been described by Lee and Parks in 1995 [5], and remain relevant in recent research [11][12].

B. Static Models of Computation

By imposing additional constraints that eliminate data-dependent behavior, we obtain static MoCs. This means, among other things, that execution and memory allocation can be fully scheduled in advance. Although such models are not suitable for applications that depend on data-encoded behavior, they foster fine-grained optimizations that are not feasible with dynamic MoCs.

Synchronous Dataflow (SDF) is popular due to its relative flexibility for a static MoC. By requiring that the number of tokens consumed and produced by an actor at each port remains constant across every firing, the behavior of the system becomes predictable, and an efficient schedule can be determined at compile-time. This approach eliminates the need for overhead-intensive dynamic runtimes, making it particularly advantageous for resource-constrained or real-time applications.

A further constraint to SDF is Single-Rate SDF, where all edges have matching input and output rates; this means that all actors fire at the same data rate [13]. Any SDF graph can be converted into a single-rate graph by duplicating actors and inserting fork/join actors to distribute the token flow between the copies; because this transformation simplifies the memory and execution dependencies of the actor firings, it is a common strategy as an intermediary step for SDF schedulers [14]. An example of this is shown in Fig. 2. However, this approach has drawbacks, covered in section IV.

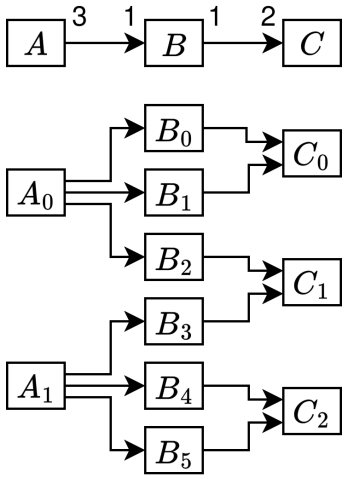

Fig. 2. Conversion from multi-rate SDF into single-rate. The number of nodes grows based on the lowest common multiple of the rates.

Fig. 1 illustrates an example of an SDF graph implementing a smoothing filter. During each firing, the *Avg3* actor computes the mean of the three input values, sends the result to the output node, and sends copies of the two most recently read values back into its feedback port for use in the next firing. These values are stored in the FIFO queue associated with the self-loop, which must have capacity for storing at least two values. FIFOs that form loops require initialization with specific values (referred to as "delays") to ensure that dependent nodes can execute during the first firing.

Not every rate configuration yields an admissible graph — that is, one guaranteed to avoid input starvation and deadlocks. Lee and Messerschmitt [15] describe a method to determine if a periodic schedule is possible, by analyzing the topology matrix, and provide an algorithm to efficiently generate both single- and multi-threaded schedules. There is generally a trade-off between period length and edge buffer size (referred to as *throughput-buffering trade-off*), as longer periods allow for more size-efficient routing when the data rates on each edge do not match nicely.

The generation of a correct maximum-throughput multi-core schedule for dataflow graphs is an exponentially complex problem [16], [17]; added to the fact that the latency of actor execution may be inconsistent and platform-dependent, this has led most dataflow compiler implementations to rely on heuristics and best-effort scheduler strategies.

The *Parameterized/Interfaced Dataflow Meta Model (PiMM)* [18] extends dataflow MoCs with features that aim to improve code reuse and modularity. *Parameters* are statically-known values in terms of which the rates of actor ports can be defined, allowing the construction of algorithms that are generic to compile-time-known constants (image sizes, for instance). *Interfaces* consist of the definition of input and output ports for a graph, allowing them to be used as actors themselves; this gives the programmer the ability to define algorithms hierarchically and generically, reducing repetition.

Applying PiMM to SDF produces the *piSDF* (or πSDF) MoC. To allow the analysis of graphs defined with these features, they are typically flattened into a standard SDF graph as part of the scheduling process. This is analogous to template instantiation, constant propagation and function inlining in procedural language compilers.

The canonical implementation of piSDF, Spider [19], employs parameters to achieve reconfigurability. Instead of treating parameters as perpetually constant values, the runtime is able to adapt to sporadic changes, generating a new schedule as needed. This restrained form of dynamic behavior is useful for use cases with variable-sized inputs, such as image processing and compression algorithms, while preserving throughput.

III. Related works

IaRa [20] is a dataflow compiler based on the MLIR framework [21], an initiative of the LLVM project [22] intended as a more modular and generic successor to the LLVM Intermediate Representation (IR). IaRa leverages the ability of MLIR to retain information from several stages of the compilation pipeline to access novel optimization opportunities, in particular where it comes to memory management, while also preserving interoperability to the state-of-the-art in the field of compilation. The contribution of this paper is implemented as a scheduling mode for IaRa.

Orcc [9] is a runtime for the RVC-CAL dataflow language, focusing on video codec implementation. As a dynamic MoC, it supports data-dependent behavior, and as such relies on runtime scheduling and memory management. In particular, Orcc requires that the programmer manually specify the sizes for each of the ring buffers associated with the graph edges, and uses scheduling heuristics to optimize for data locality and to reduce context switching.

Preesm [14] is a dataflow framework geared for rapid prototyping of DSP programs. It includes an Eclipse-based graphical interface for visual definition of graphs, and an optimizing compiler that consumes as input topological descriptions of the dataflow graph and the target architecture and outputs C files containing the precomputed schedule, to be linked to actor implementations with a conventional C/C++ compiler. It is bundled with Spider [19], a runtime for the piSDF MoC that is intended for the development of parallel programs on DSP processors, being presented as a direct alternative to OpenMP.

Preesm relies on C generation as its choice of IR, and on conversion to single-rate dataflow.

Taskgraph [23] is an optimization to the OpenMP tasking framework that reduces the synchronization overheads of static task Directed Acyclic Graphs (DAGs) by recording and replaying valid schedules. While the project has found success in improving performance in suitable applications, it is limited to single-rate acyclic graphs.

IV. Problem Statement

A. Limitations of static scheduling

Static scheduling has remained the state of the art in SDF for a long time. This focus is justified [24], considering that many SDF-based applications focus on latency and memory footprint, for which static scheduling appears to be the most effective solution. The main strategy for the scheduling of SDF is conversion to single-rate graphs[25]; this allows scheduling to happen individually for each firing in a graph iteration, with fine-grained control over thread assignment and memory access. Preesm [14], for instance, employs techniques such as memory exclusion graphs [26] at compile time to provide an upper bound on total memory usage. The fact that memory offsets and processor assignments are precomputed and stored in the binary as part of the schedule means that these decisions do not have to be made at runtime, and the lack of ad-hoc memory operations means that latency and throughput can be more easily predicted [27].

While this approach of conversion to single-rate allows powerful scheduling techniques for a wide variety of applications, it struggles with scalability; depending on the graph topology and the ratios between token rates, the size of the single-rate equivalent graph may increase exponentially, impacting compilation time and binary size directly as each firing must be individually represented in the final precomputed schedule. This is the case even in highly regular workloads such as tensor operations; an element-wise operation into an $N \times N$ matrix corresponds to a static schedule with size and compilation time proportional to N^2, while at the same time discarding information about regularity and symmetry of the computation that could otherwise be useful in the lower-level stages of the analysis and optimization pipeline; in such cases, this prevents this type of scheduler from being competitive with conventional approaches in the shared memory landscape, such as polyhedral optimization of affine loops [28].

This property makes it challenging to use static scheduling for situations that require dynamic recompilation [29]; for large enough token ratios, it becomes infeasible to compile even in environments that are not resource-constrained. Remedying this phenomenon is a continuing focus of research, with strategies such as compact graph representation [30] and actor clustering [31].

In the meantime, a robust solution for these cases is to rely on dynamic scheduling. Dynamic dataflow MoCs such as DPN and KPN are supersets of SDF, meaning that their runtimes can execute SDF graphs with little adaptation effort. However, since such schedulers are built to be robust to the unpredictable nature of Turing-complete systems, they do not typically leverage static SDF information for optimization.

B. Memory management in dynamic runtimes

For dynamic scheduling, the most common memory management strategy is the ring buffer, an example of which is shown in Fig. 3. It maintains the first-in-first-out (FIFO) semantics of dataflow, guaranteeing the logical order of operations and memory access between producer and consumer nodes. It consists of a contiguous buffer of memory and two access pointers, pointing to the front and back of the queue. Both insertion and removal cause the pointers to increment, with modulo semantics; once they reach the end of the buffer, the pointers wrap around to the beginning. Because of this, ranges of tokens are not necessarily contiguous, as they may cross this wraparound boundary.

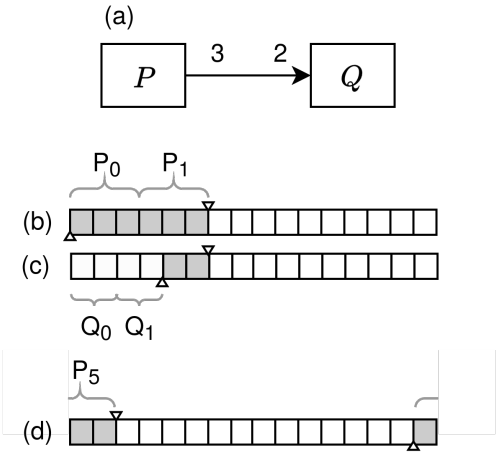

Fig. 3. Example of ring buffer operation. (a) An SDF edge with a 3:2 data ratio. (b) Producer firings insert new tokens in the free space available at the "back" of the queue (▽). (c) Consumer firings remove tokens from the "front" of the queue (△). (d) Token spans may cross a wrap-around boundary.

Fixed-size ring buffers are common in implementations of static dataflow systems for specialized architectures, such as FPGAs, DSPs and scratchpads [32], because they bypass the overheads of parallelism and caches in shared memory architectures. The problem of determining a size for each ring buffer is complex, as the total buffer size depends on the maximum amount of tokens held by each edge during the execution of the program. This is usually tackled either at compile time (automatically, together with other analysis) [13] or manually, through trial and error. Increasing the size of the buffer gives more flexibility to the scheduler, at the price of increased memory footprint.

Conventional implementations of ring buffers only expose the first and last items of the FIFO at a time, preventing more than one firing of an actor to execute simultaneously. This is a typical choice in dynamic dataflow; in ORCC[33], for instance, the scheduling strategy is to run the same actor as many times as possible on the same PE, until the output buffers become full or the input buffers become empty; only task parallelism can be exploited, by assigning different actors of the graph to different PEs.

Some specialized implementations overcome this limitation by allowing actors to "skip the line", using externally-provided buffer and sizing information to items in the middle of the buffer, which re-enables some data parallelism. This is more common in static dataflow implementations for specialized memory architectures such as FPGAs, DSPs and scratch-

pads [34].

Where it comes to memory access by the actor implementations, schedulers must choose one of two strategies, here arbitrarily labeled 1 and 2. *Strategy 1* consists of copying the tokens to an intermediate location, removing them from the ring buffer *before* allowing the actor to read it. While this choice requires extra copies and memory, it unencumbers the schedule, freeing that section of the buffer to be reused without having to wait for the consumer actor to finish the firing. This also provides an opportunity to realign the data if it is crossing the ring buffer wrap-around boundary (as in Fig. 3(d)), retaining compatibility with actors that expect memory contiguity, and also preventing destructive cache interference for the parallel case. *Strategy 2* consists of allowing the actor to read directly from the ring buffer, saving memory operations but consuming space in the buffer for the duration of the firing. This also requires that the actor be implemented with knowledge of the scheduler, as the size and location of the ring buffer is necessary for the modulo operation. This also prevents the leveraging of existing kernel implementations that expect contiguous buffers, such as the ones provided by BLAS [35], [36], [37].

Dynamic MoCs, being Turing-complete, do not have a provable upper bound on the amount of tokens held by each edge at a given point in time, which makes fixed-sized ring buffers an unsuitable choice when optimizing for memory footprint. In this case, the conventional choice is the self-growing ring buffer: when trying to insert a new value into a ring buffer that is already full, it dynamically allocates a larger buffer and copies itself into it (usually doubling in size each time, leading to an $O(\log n)$ number of allocations). While this is enough to prevent deadlocks, it does not provide a way to release unused memory back to the system, making the strategy sensitive to graph irregularity.

V. Target contributions

Considering the limitations of the ring buffer approach and keeping in mind the SDF semantics leveraged by static schedulers, we arrive at our novel "*quasi-dynamic*" integrated scheduling and memory management system, described as such because it leverages static information to mitigate the overhead of dynamic runtimes while retaining their benefits. In contrast to the *quasi-static* MoCs [8] (mostly static systems with dynamic exceptions), we define a *quasi-dynamic runtime scheduler* as a runtime scheduler with graph-specific optimizations generated at compile time.

It consists of two parts, both of which rely on static dataflow information to function:

- The *virtual FIFO*, a memory allocation scheme where memory is allocated in small parcels (*blocks*), as opposed to large contiguous spans. The allocation sizes of these are derived from dataflow rates.
- A *keyed semaphore* self-timed synchronization system based on concurrent hash maps, leveraging static information about the total number of times a given memory region will be accessed to efficiently reconstruct dependencies at runtime.

The virtual FIFO retains the semantics of a ring buffer, while improving the flexibility of the scheduler. The relationship between the firings of the producer and consumer nodes is preserved; within a single-rate edge, for instance, the token that was written by a given firing of the producer node is guaranteed to be read by the corresponding firing of the consumer node. However, virtual FIFOs lack the aforementioned downsides of ring buffers. In particular, dependencies between different firings of an actor are decoupled from the memory they access, allowing less constrained operation of the underlying task scheduler and heap allocator.

One caveat of virtual FIFOs is that they attain increased parallelism by sacrificing the memory locality afforded by ring buffers. In contexts with low PE counts and highly regular fine-grained graphs (where many sequential firings are likely to be assigned to the same PE), large ring buffers are expected to have superior throughput, as memory prefetchers operate optimally with sequential memory access. As such, virtual FIFOs are better suited to coarse-grained graphs, or to memory-constrained applications.

A. Virtual FIFO scheduling

Like a self-growing ring buffer, virtual FIFOs also allocate memory as needed. Differently from ring buffers, the regions of memory being accessed by different firings of a node may be allocated independently, potentially far apart in the address space, and not necessarily in the same chronological order. Additionally, all edges share the same pool of free memory, which is allocated as the input tokens of the producer node become available and freed as the consumer finishes executing.

This also obviates the need for explicit buffer sizing; memory that is not currently storing a token is available to every other FIFO in the graph.

Memory is stored in the form of independently allocated regions of memory called *blocks*, sized in such a way as to fit the access patterns of the consumer and producer node without causing unnecessary copies and alignment corrections.

In lieu of tokens, actors communicate by references to contiguous subsets of blocks, termed *slices*. Allocation and deallocation nodes are generated at compile time; allocation nodes provide blocks of the appropriate size to be accessed directly by the actors without copy or alignment operations; deallocation nodes simply wait for ownership of all slices of a block to be returned before freeing the entire block, in a process analogous to reference counting.

Each slice contains:

- A pointer and size to the relevant subset of the block;
- A *virtual offset*; this value is unique for each slice and indicates the sequence number of the firings that process it. It starts at zero with the first allocation of the program and increases with each new block inserted in the FIFO, as if it were an index into an array of arbitrary size. The name "virtual" is inspired by the virtual addressing

capabilities of modern shared memory architectures, but does not rely on them.

Given the virtual offset of a given slice, the statically known data about delays and token rates, and the calculated block sizes for each edge, it is possible to reconstruct, at runtime, which block portions are to be processed by which firing of the consumer actor, and its execution can be scheduled accordingly.

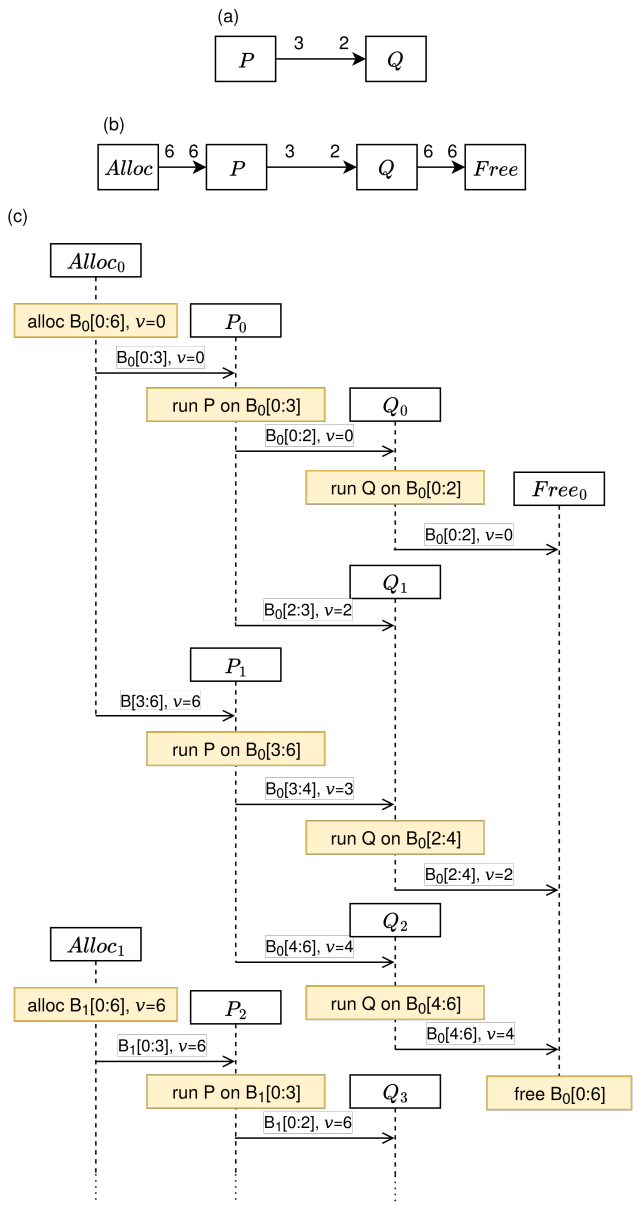

Fig. 4. Example of operation of virtual FIFO. (a) Original graph (b) Graph with inserted allocation and deallocation nodes, sized so that it fits 2 executions of P and 3 executions of Q. (c) On-line scheduling: for each FIFO, firings of the producer node calculate the downstream dependents of the produced slice. v stands for the virtual offset of each slice. White boxes represent actor firings, and yellow boxes represent the moment of their execution.

A step-by-step example showing the behavior of the virtual FIFO memory allocation during the execution of a multi-rate SDF graph is shown in figure 4. Here, the Python-like notation [0:3] indicates a slice that references the first 3 tokens of the 6-token block B_0. Once firing P_0 has finished processing slice [0:3], it deduces that it is a dependency of Q_0 and Q_1, forwarding them sub-slices [0:2] and [2:3], respectively. Since the size of [0:2] matches the input rate of Q, Q_0 may be scheduled right away; conversely, Q_1 must wait for the execution of P_1, and so P_0 notifies the synchronization system that it has provided 1 of the 2 required tokens. Once P_1 executes, it will provide the remaining token to the semaphore, which will schedule Q_1. The process continues forward until all slices reach the $Free$ node, which reconstructs the original allocation pointer and returns it to the heap allocator.

Note that horizontal lines intersecting a vertical dashed line represent concurrent events that may occur in any chronological order. It is possible, for instance, that P_1 executes before P_0. Note also that the value of v starts at 6 in the block generated by $Alloc_1$; $Alloc_2$ would have $v = 12$, $Alloc_3$ would have $v = 18$, and so on, allowing them to retain correct behavior even if they execute out of order.

An advantage of this system is the fact it is naturally compatible with the in-place ports extension of the SDF model, which is explored as a copy elision optimization in the Preesm compiler by Desnos et al. [38]. If a given pair of input and output ports is known to refer to the same memory, we may avoid copying it to a newly allocated buffer provided that alignment constraints are respected along the entire chain of in-place edges (Fig. 6).

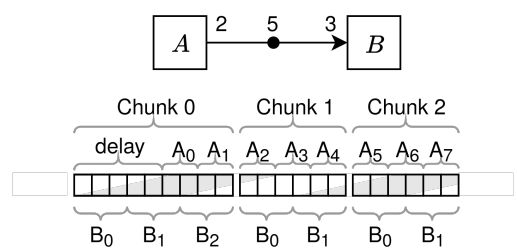

Fig. 5. Memory alignment with delays and allocations ("blocks"). The first block must contain space for the delay values, while subsequent blocks do not. Note that allocations do not necessarily map one-to-one to the formal dataflow graph iterations (shown by the alternating shaded regions); more than one iteration may access the same block and vice-versa. Because actor firings may occur out of order, the execution timing of different graph iterations may overlap.

Figs. 5 and 6 show examples of buffer sizing for edges with different token rates and delay sizes; the objective is to allow all actors in the chain to access the same buffer without adding unnecessary memory operations or scheduling dependencies. Because each firing expects memory contiguity, the chosen size must fit a whole number of firings of each actor that accesses that buffer, accounting for the initial delays, such that no firing crosses an allocation boundary. Due to these

alignment constraints, valid buffer sizes must correspond to integer solutions to the system in Equation 1 (because we are only interested in integer solutions, these are first-order Diophantine equations [39]). For simple edge chains with no delays, this size consists of the least common multiple of the token rates (as in Fig. 6(c). For edges with delays, the first block has a different size that accounts for the correct alignment of the delays (Fig. 6(b)).

$$\begin{aligned} D_{ij} + R_i\alpha_i = R_j\alpha_j \quad \forall\ e_{ij} \in E \\ R_i\beta_i = R_j\beta_j \quad \forall\ e_{ij} \in E \end{aligned} \tag{1}$$

In Equation 1, for each edge e_{ij} in a read-write chain E, D_{ij} is the size of its delay and R_i and R_j are the rates of the producing and consuming actors, respectively. We are left with positive integer variables α and β to optimize for each input/output port pair, representing the number of firings its actor will execute on a block of that size, in the cases with and without the initial delays, respectively.

As shown in Fig. 4, a read-write chain is at least 4 actors long (the allocation, the writing by the first actor, the reading by the second actor, and the freeing). Longer chains of actors that operate on the same memory insert additional constraints to the alignment, and therefore affect the size of the block, particularly when rates and delay sizes share no prime factors. In the example of Fig. 6, the smallest block size that guarantees alignment of all data accesses is 30, for both delay blocks and non-delay blocks. Values for D and R represent the sizes of the delays and data rates, respectively, and are obtained directly from the graph. The compiler must provide values for α and β, representing the number of firings that each actor executes on a block with and without delays, respectively. This can be done, for instance, through simple integer linear programming (ILP). While a solution based on Hermite normal form matrices would be technically more efficient [40], in practice the ILP approach appears to suffice for the size and rates in the read-write chains of realistic applications.

A caveat to note is that there are combinations of rates and delays that lead to no solutions, such as the one shown in Fig. 7. In such a case, there is no value of α that will cause an alignment of all memory access boundaries. In such a case, the compiler will fail and a copy must be inserted.

B. Keyed semaphore

While there exist "lockless" strategies for a selection of parallel algorithms, they can be challenging to implement correctly and are not guaranteed to exist for a given problem; as such, the most widespread practice in the development of parallel algorithms in shared memory contexts is through the use of synchronization locks (mutexes, semaphores, etc).

One of the main challenges faced by runtime schedulers is the overhead caused by locking [41]. This is one of the main advantages of static scheduling, as the need for thread synchronization is reduced when actor firings are assigned to PEs ahead of time.

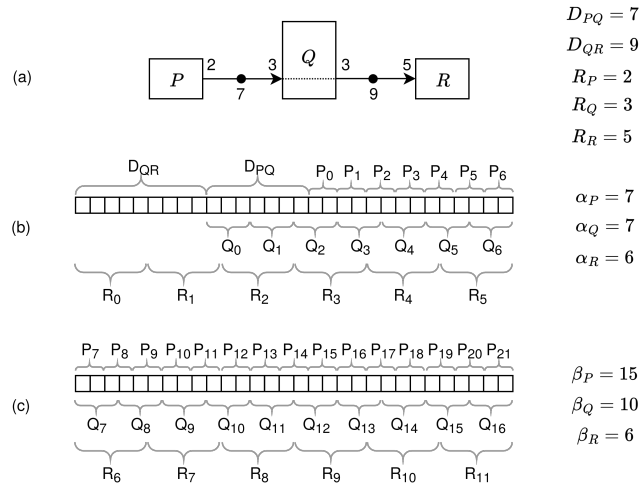

Fig. 6. Example of memory allocation with multi-rate edges, and corresponding values to Eq. 1. (a) Source SDF graph (Q operates in-place, represented by dotted line.) (b) First allocation accounts for initial delay. (c) Second allocation and onwards do not need to account for the delay.

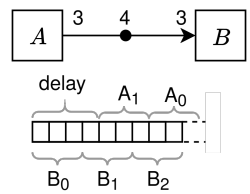

Fig. 7. Example of graph with no solutions for allocation of the first block; there are no values of α that will lead to an alignment of A and B.

Runtime schedulers, on the other hand, require on-the-fly assignment. While they cannot be completely avoided, it is possible to design a scheduler in such a way as to minimize locking. Ideally, synchronization should only happen between tasks that logically depend on each other, which means having a separate lock for each synchronization point. With a static SDF schedule, the binary footprint is already proportional to the size of the expanded single-rate graph; allocating locks statically, even naively, does not significantly impact the binary size.

We are aiming for a binary size proportional to the size of the original multi-rate graph; since it is smaller than the expanded graph that the scheduler will reconstruct at runtime, this necessarily means sharing memory space for the locks while they are not being used.

While we use OpenMP for the underlying parallel task management, its tasking framework is unsuitable for this case; while OpenMP dependencies have associated locks, they are declared through its source code annotation system, and exist individually in memory. Furthermore, OpenMP does not provide a mechanism to synchronize events whose execution

order is completely unknown at compile time, which is the case for our completely self-timed scheduler [15].

To address this, we introduce a new data structure, the *keyed semaphore*. It consists of a collection of parallel hash maps, one for each actor in the SDF graph, keyed by the actor sequence number so that each firing has a unique identifier. Unlike a static schedule, where each effective firing in a graph iteration has associated static information, each entry in the keyed semaphore only exists while it is being used: from the moment one of the actor dependencies first finishes executing, until the firing of the actor itself. Our implementation takes advantage of SDF analysis to reuse the locks after the synchronization is realized.

Each entry contains a resource counter and a list of slices, representing pointers to the tokens that will be consumed. As they finish executing, each dependency of a firing looks for the existence of the corresponding entry in the hash map. The first one to arrive, finding that the entry does not exist, will initialize it with the total input size of the consumer actor (the sum of all its input rates). As each subsequent dependency "arrives", it populates the slice list and decrements the resource counter, according to the size of its contribution. When the resource counter reaches zero, the firing can be scheduled with the complete slice list and the entry can be destroyed.

The "naive" implementation of a keyed semaphore consists of a standard hash map, protected by a single mutex. However, such an implementation would cause considerable contention, particularly for fine-grained graphs with many iterations per actor, as all iterations would wait on the same lock. To avoid this problem, an off-the-shelf parallel hash map implementation is used [42]. While it does not completely remove the overhead of synchronization, it mitigates it by segmenting the hash map into several parts, each protected by a separate lock, therefore decreasing the likelihood of two threads attempting to acquire the same lock.

C. Runtime implementation

The current proof-of-concept implementation of the scheduler leverages off-the-shelf implementations of heap allocation, runtime task scheduling and concurrent maps (respectively, `libc malloc`, OpenMP Tasks and `gtl/phmap`). While they are sufficient for correct execution, there is clear potential for optimization by specializing these algorithms for the context of SDF. The source code for IaRa is available at `https://github.com/jabcross/iara`.

VI. Experiments

Through our experiments, we aim to achieve the following objectives:

1) Demonstrate a feasible and robust scheduler that covers use-cases for which static scheduling is very time costly and generates too large binaries;
2) Show limited latency overhead in comparison to conventional static scheduling.

For objective 1, we compare the results of our scheduler against Preesm. Our chosen application is the *degridder* step of the DDFacet [43] radio astronomy simulator.

In this section, we go into the details of our chosen target application, explain the differences of the Preesm and IaRa implementation, and show the results of experimentation over the parameter space and target measurements.

A. The degridder application

In the field of radio astronomy, *gridding* is a data processing procedure that consists of interpolating the irregularly-spaced *visibility samples* of a radio interferometer (such as those produced by individual antennae in an antenna array) into a regular grid, in order to convert them into an image [44]. The reverse operation, *degridding* — transforming an image of the sky into the measurements that *would* have been produced by the same instruments — is an essential step in the calibration of such devices.

Both processes involve heavily data-centric operations such as convolutions and Fourier transforms, and the size of real-life datasets involved is large, ranging from gigabytes to petabytes. Because the only practical way to process such large datasets is through streaming, and the operations are largely data-independent, this application lends itself naturally to dataflow as an implementation strategy.

The implementation we use as reference is itself adapted from the DDFacet Imaging Project [43], originally implemented procedurally in Python with the NumPy library. The dataflow implementation is developed in Preesm, in the piSDF (piSDF) MoC. The datasets used are simulated measurements made available by the Square Kilometer Array (SKA) Observatory in their GitHub repository [45].

1) Preesm implementation: The reference implementation is developed in Preesm, using the standard piSDF workflow and C++ actor implementations. It uses the hierarchical features of piSDF to separate the parallelized region (an implementation of the *degridder* kernel that separates the data into independent chunks) from the data I/O operations, and parameterizes it over the properties of each dataset and the chosen granularity. As Preesm allows manually setting the PE affinity of each actor individually, the original implementation constrains the IO actors to a single core, only allowing the degridding actor access to the entire set. Because IaRa does not provide this feature, and in order to ensure fair comparison, this constraint is removed for our experiments.

2) IaRa implementation: Our implementation consists of a direct translation of the Preesm graphs into the IaRa dataflow dialect, and measure with both the virtual FIFO and ring buffer schedulers. We use the same C/C++ kernel implementations as the Preesm version does, with minimal alterations to remove Preesm-specific code.

B. Experimental parameters and measurements

The SKA project provides three reference datasets, of small, medium and large sizes (amounting to 3,924,480, 7,848,960 and 31,395,840 visibility samples, respectively). We limit our

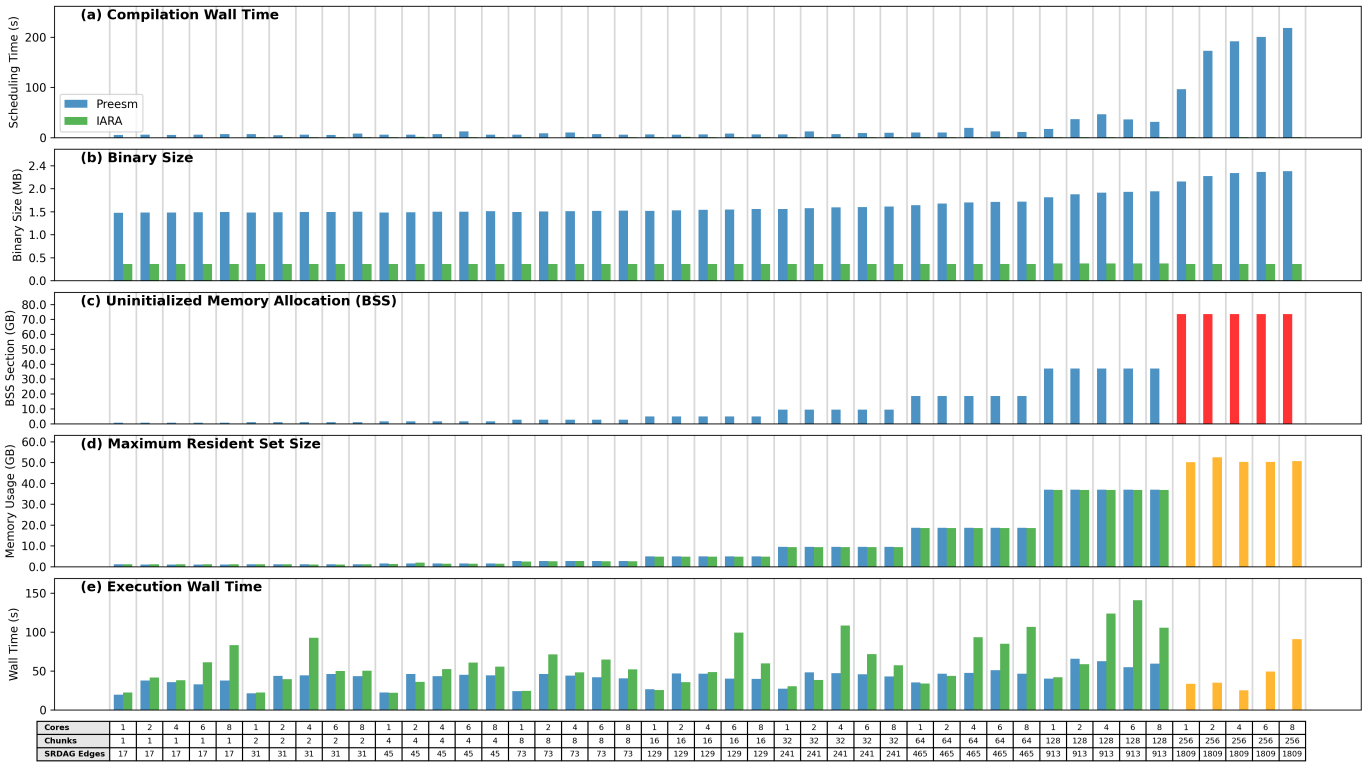

Fig. 8. Comparison between Preesm and IaRa for different instances of the degridder application. Red bars indicate instances where the Preesm version failed to run due to not enough memory being available. Orange bars indicate instances where the IaRa version was terminated by the OS due to running out of memory.

experiments to the large dataset. We also parameterize the system parallelism (amount of cores available to the scheduler) and granularity (the amount of chunks each image is split into to be processed by individual actors).

Through our experiments, we measure compilation time, binary size, maximum memory usage of the executable, and execution wall time. We also provide the number of nodes and edges of the single-rate graph expansion generated by Preesm.

The experiments are executed on a laptop with an Intel Core Ultra 5 125H (an heterogeneous architecture containing 4 "performance" cores and 10 "efficiency" cores) and 64GB of RAM. To ensure consistency in testing, the experiments are limited to the first 8 hardware threads, which execute exclusively on the performance cores.

VII. Results

In this section, we examine the results of the compilation and execution experiments between Preesm and IaRa, looking at binary size, compilation time, wall time, runtime memory consumption and static memory allocation. Fig. 8 combines these measurements with a table showing number of cores, granularity, and the number of edges of the expanded single-rate graph.

A. Compilation wall time

One of the advantages of runtime scheduling is the relatively low computational complexity of the compilation process. Here, we only measure the wall time of the compilation of the schedule (Preesm's C code generation and IaRa's LLVM IR generation). Counting only the successful instances (up to and including the 128-chunk instances), IaRa achieves an average speedup of 12.2x.

Fig. 8(a) shows the time required for Preesm to generate a schedule. As the number of edges in the expanded single-rate graph grows, so does compilation time; this growth also shows some non-linearity. Not shown in the graph are instances of chunk count 256, 512 and 1024, as Preesm fails to finish scheduling of these instances within a five-minute timeout.

The compilation times for IaRa barely show in the graph. This is because the work done at compile time depends only weakly on the rates of the graph and does not scale in proportion to the single-rate expansion. Compilation times for the runtime do not exceed 1.5 seconds.

B. Execution wall time

Lastly, we compare the performance of Preesm with the runtime implemented in IaRa (Fig. 8(e)). The wall time remains in the same order of magnitude as Preesm's, even considering the unoptimized state of IaRa. The overall average slowdown

of IaRa is of 1.43x. Measuring the average separately based on core count, we get the values in Table I. We can see clear correlation between the number of cores and the overhead in the IaRa runtime. It is noteworthy that the two-core instances do not show slowdown; this may be explained by the fact that both threads execute in the same physical core through Intel's superscalar implementation (hyperthreading), reducing the impact of context switching.

TABLE I
AVERAGE IARA EXECUTABLE SLOWDOWN W.R.T. PREESM, BY CORE COUNT

# Cores	Avg. slowdown
1	1.03
2	0.96
4	1.63
6	1.77
8	1.61
All instances	1.43

If we interpret that the difference between the runtimes of Preesm and IaRa consists of the scheduling work being done at runtime by IaRa, we may compare the total time spent on scheduling for each compiler, including both compile-time and runtime. For dynamic systems where graphs must be reconfigured on-site, the fact that the static compilation time increases exponentially is evidence that runtime scheduling could be worth it in terms of total CPU time and of reconfiguration latency, depending on the nature of the algorithm and the frequency of reconfiguration.

C. Memory consumption

One detail to note is that the effective static memory used by the program does not necessarily match the binary size; the program may request uninitialized or zero-initialized static memory from the OS. This is the strategy adopted by Preesm, as can be seen in the *block starting symbol* (BSS) section plot (Fig. 8(c)). Starting with the 256-chunk instances, the Preesm version fails to start altogether, as the generated schedule attempts to reserve more static memory than is available on the system. Conversely, the IaRa version starts execution and requests memory from the system as it is needed, only being killed by the operating system once memory runs out.

One would not expect the memory usage to increase this much just as a consequence of increasing granularity. In this case, this is caused by more complex cases of automatic copy elision than currently supported by each compiler, and the fact that they do not take into account the system's memory availability when scheduling.

Fig. 8(d) shows the maximum memory consumption for each instance, as reported by the *Maximum Resident Set Size* (RSS) measurement provided by the GNU `time` tool. For the instances that execute successfully, they are almost identical; IaRa achieves 1.007x smaller sizes. We attribute this difference to the binary size itself, as this measurement relates to the maximum amount of memory pages allocated by the OS to each process, with a granularity of 4096 bytes.

D. Binary size

Both Preesm and IaRa rely on an external C/C++ compiler for the compilation of the kernel implementations. As part of the process, they both emit an object file containing the scheduling information; the precomputed schedule in the case of Preesm, and the runtime scheduler itself in the case of IaRa. This object file is linked against the implementation of the actors, which is the same in both cases. In Fig. 8(b), we plot the binary size of the final executable emitted by each compiler; because the kernels take a considerable amount of space (due to the libraries accessed by the kernels), the impact of the scheduler is made more clear by showing the contribution of the scheduler to the total size (measured by the size of the object file generated by each compiler). We see that the precomputed schedule itself takes considerable space in the Preesm binaries, and that its size changes with the size of the expanded graph. Conversely, the sizes of the IaRa binaries are on average 4.4x smaller, and do not vary based on the graph parameters.

VIII. CONCLUSION

In this work, we have demonstrated that there is space for research on runtime scheduling for static dataflow MoCs. Our proof-of-concept quasi-static scheduler shows improved scalability in comparison to static scheduling, even in its current unoptimized state, keeping constant, parameter-independent compile times while retaining or decreasing memory usage; future versions will tackle synchronization overheads, automatic memory optimization and delegation of the problematic high-granularity cases to specialized tools such as polyhedral compilation.

REFERENCES

[1] S. S. Bhattacharyya, E. F. Deprettere, R. Leupers, and J. Takala, "Handbook of signal processing," in *Handbook of Signal Processing Systems*, 3rd ed. Springer, 2019.

[2] E. A. Lee and S. Ha, "Scheduling strategies for multiprocessor real-time DSP," in *Global Telecommunications Conference and Exhibition 'communications Technology for the 1990s and beyond' (GLOBECOM), 1989. IEEE*, 1989, pp. 1279–1283.

[3] M. Herlihy, N. Shavit, V. Luchangco, and M. Spear, "The art of multiprocessor programming." Elsevier, 2021, ch. 13.

[4] G. F. Zaki, W. Plishker, S. S. Bhattacharyya, and F. Fruth, "Implementation, scheduling, and adaptation of partial expansion graphs on multicore platforms," *Journal of Signal Processing Systems*, vol. 87, no. 1, pp. 107–125, 2017.

[5] E. A. Lee and T. Parks, "Dataflow process networks," *Proceedings of the IEEE*, vol. 83, no. 5, pp. 773–801, 1995.

[6] J. L. Kelly, C. Lochbaum, and V. A. Vyssotsky, "A block diagram compiler," *The Bell System Technical Journal*, vol. 40, no. 3, pp. 669–678, 1961.

[7] W. R. Sutherland, "The on-line graphical specification of computer procedures," Ph.D. dissertation, Massachusetts Institute of Technology / Massachusetts Institute of Technology, 1966.

[8] G. Roumage, S. Azaiez, C. Faure, and S. Louise, "An extended survey and a comparison framework for dataflow models of computation and communication," *arXiv preprint arXiv:2501.07273*, 2025.

[9] H. Yviquel, A. Lorence, K. Jerbi, A. Sanchez, G. Cocherel, and M. Raulet, "Orcc: Multimedia development made easy," in *Proceedings of the 21st ACM International Conference on Multimedia*, 2013, pp. 863–866.

[10] G. Kahn, "The semantics of a simple language for parallel programming," *Information processing*, vol. 74, pp. 471–475, 1974.

[11] O. Rafique and K. Schneider, "Synthesis of parallel software from heterogeneous dataflow models," *SN Computer Science*, vol. 3, no. 3, p. 249, 2022.

[12] K. Schneider, "Translating structured sequential programs to dataflow graphs," in *Proceedings of the 19th ACM-IEEE International Conference on Formal Methods and Models for System Design*, 2021, pp. 66–77.

[13] S. S. Bhattacharyya, P. K. Murthy, and E. A. Lee, "Synthesis of embedded software from synchronous dataflow specifications," *Journal of VLSI signal processing systems for signal, image and video technology*, vol. 21, no. 2, pp. 151–166, 1999.

[14] M. Pelcat, K. Desnos, J. Heulot, C. Guy, J.-F. Nezan, and S. Aridhi, "Preesm: A dataflow-based rapid prototyping framework for simplifying multicore DSP programming," in *Education and Research Conference (EDERC), European Embedded Design In*, Sep. 2014, pp. 36–40.

[15] E. Lee and D. Messerschmitt, "Synchronous data flow," *Proceedings of the IEEE*, vol. 75, no. 9, pp. 1235–1245, 1987. [Online]. Available: http://ieeexplore.ieee.org/document/1458143/

[16] A. Ghamarian, M. Geilen, T. Basten, B. Theelen, M. Mousavi, and S. Stuijk, "Liveness and boundedness of synchronous data flow graphs," in *2006 Formal Methods in Computer Aided Design*, 2006, pp. 68–75.

[17] S. Stuijk, M. Geilen, and T. Basten, "Throughput-buffering trade-off exploration for cyclo-static and synchronous dataflow graphs," *IEEE Transactions on Computers*, vol. 57, no. 10, pp. 1331–1345, 2008.

[18] K. Desnos, M. Pelcat, S. S. Bhattacharyya, and S. Aridhi, "PiMM: Parameterized and interfaced dataflow meta-model for MPSoCs runtime reconfiguration," in *Embedded Computer Systems (SAMOS), 2013 International Conference On*, 2013.

[19] J. Heulot, M. Pelcat, K. Desnos, J.-F. Nezan, and S. Aridhi, "SPIDER: A synchronous parameterized and interfaced dataflow-based RTOS for multicore dsps," in *Education and Research Conference (EDERC), 2014 6th European Embedded Design In*, 2014, pp. 167–171.

[20] P. Ciambra, M. Dardaillon, M. Pelcat, and H. Yviquel, "Co-optimizing Dataflow Graphs and Actors with MLIR," in *2022 IEEE Workshop on Signal Processing Systems (SiPS)*. Rennes, France: IEEE, Nov. 2022, pp. 1–6. [Online]. Available: https://ieeexplore.ieee.org/document/9919213/

[21] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "MLIR: Scaling compiler infrastructure for domain specific computation," *International Symposium on Code Generation and Optimization (CGO)*, pp. 2–14, 2021.

[22] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *International Symposium on Code Generation and Optimization (CGO)*, no. c, 2004, pp. 75–86.

[23] C. Yu, S. Royuela, and E. Quinones, "Taskgraph: A low contention openmp tasking framework," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 8, pp. 2325–2336, 2023.

[24] I. Hautala, J. Boutellier, T. Nyländen, and O. Silvén, "Toward efficient execution of rvc-cal dataflow programs on multicore platforms," *Journal of Signal Processing Systems*, vol. 90, no. 11, pp. 1507–1517, 2018.

[25] R. de Groote, P. K. F. Holzenspies, J. Kuper, and H. Broersma, "Back to Basics: Homogeneous Representations of Multi-Rate Synchronous Dataflow Graphs."

[26] K. Desnos, M. Pelcat, J.-F. Nezan, and S. Aridhi, "Distributed Memory Allocation Technique for Synchronous Dataflow Graphs," in *2016 IEEE International Workshop on Signal Processing Systems (SiPS)*, Oct. 2016, pp. 45–50. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7780070

[27] A. H. Ghamarian, M. C. W. Geilen, S. Stuijk, T. Basten, B. D. Theelen, M. R. Mousavi, A. J. M. Moonen, and M. J. G. Bekooij, "Throughput analysis of synchronous data flow graphs," in *Sixth International Conference on Application of Concurrency to System Design (ACSD'06)*, 2006, pp. 25–36.

[28] W. S. Moses, L. Chelini, R. Zhao, and O. Zinenko, "Polygeist: Affine c in mlir," 2021.

[29] M. Pelcat, J.-F. Nezan, and S. Aridhi, "Adaptive multicore scheduling for the LTE uplink," in *2010 NASA/ESA Conference on Adaptive Hardware and Systems*, Jun. 2010, pp. 36–43. [Online]. Available: https://ieeexplore.ieee.org/document/5546233/

[30] F. Arrestier, K. Desnos, E. Juarez, and D. Menard, "Numerical Representation of Directed Acyclic Graphs for Efficient Dataflow Embedded Resource Allocation," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 18, no. 5s, pp. 101:1–101:22, Oct. 2019. [Online]. Available: https://doi.org/10.1145/3358225

[31] O. Renaud, H. Miomandre, K. Desnos, and J.-F. Nezan, "Automated level-based clustering of dataflow actors for controlled scheduling complexity," *Journal of Systems Architecture*, vol. 154, p. 103217, 2024.

[32] E. A. Lee and S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*. Mit Press, 2016.

[33] Orcc, "Open RVC-CAL compiler : http://orcc.sourceforge.net."

[34] W. Che and K. S. Chatha, "Scheduling of synchronous data flow models onto scratchpad memory-based embedded processors," *ACM Trans. Embed. Comput. Syst.*, vol. 13, no. 1s, pp. 30:1–30:25, Dec. 2013. [Online]. Available: https://dl.acm.org/doi/10.1145/2536747.2536752

[35] C. L. Lawson, R. J. Hanson, D. R. Kincaid, and F. T. Krogh, "Basic linear algebra subprograms for fortran usage," *ACM Transactions on Mathematical Software (TOMS)*, vol. 5, no. 3, pp. 308–323, 1979.

[36] J. J. Dongarra, J. Du Croz, S. Hammarling, and R. J. Hanson, "An extended set of fortran basic linear algebra subprograms," *ACM Transactions on Mathematical Software (TOMS)*, vol. 14, no. 1, pp. 1–17, 1988.

[37] J. J. Dongarra, J. Du Croz, I. S. Duff, and S. Hammarling, "A set of level 3 basic linear algebra subprograms," *ACM Transactions on Mathematical Software (TOMS)*, vol. 16, no. 1, pp. 1–17, 1990.

[38] K. Desnos, M. Pelcat, J.-F. Nezan, and S. Aridhi, "On memory reuse between inputs and outputs of dataflow actors," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 15, no. 2, pp. 1–25, 2016.

[39] G. H. Hardy and E. M. Wright, *An introduction to the theory of numbers*. Oxford university press, 1979.

[40] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2007.

[41] R. D. Blumofe and C. E. Leiserson, "Scheduling multithreaded computations by work stealing," *Journal of the ACM (JACM)*, vol. 46, no. 5, pp. 720–748, 1999.

[42] G. Popovitch, "gtl," 4 2024. [Online]. Available: https://github.com/greg7mdp/gtl

[43] C. Tasse, B. Hugo, M. Mirmont, O. Smirnov, M. Atemkeng, L. Bester, M. J. Hardcastle, R. Lakhoo, S. Perkins, and T. Shimwell, "DDFacet: Facet-based radio imaging package," Astrophysics Source Code Library, record ascl:2305.008, May 2023.

[44] A. R. Thompson, J. M. Moran, and G. W. Swenson, Jr., *Interferometry and Synthesis in Radio Astronomy*, 3rd ed. Cham, Switzerland: Springer International Publishing, 2017.

[45] SKA Telescope, "sim-datasets," 2023. [Online]. Available: https://github.com/ska-telescope/sim-datasets